HEALTH iT · AI ENGINEERING

YacssoK(약쏙)

사진으로 찾아보는 알약 도우미

0.60981

Kaggle Public 팀 최고 점수

처음 보는 사진 842장 · YOLO26m, v3allc 기본

+0.196

같은 모델에서 데이터를 바꿔 올린 점수

YOLO26s Kaggle 0.398 → 0.594

0.99094

공식 검증 mAP75_95 최고

YOLO26s · v3allc + RFS + Copy-Paste

156 / 156

검증 알약 이름 모두 맞힘

정답 약 최저 신뢰도 0.82

팀	헬스잇(Health iT) AI 엔지니어링 팀
기간	2026.10.22 – 2026.11.06
작성일	2026.11.05
평가 지표	mAP@[0.75:0.95] (이하 mAP75_95). 이 보고서의 모든 점수는 이 지표 하나만 쓴다.
이미지 사용	대회 데이터는 재배포가 금지되어 있어, 이 보고서의 그림은 모두 숫자로 그린 그래프다.

목차

0 한눈에 보기

1 왜 YacssoK(약쏙)인가

2 서비스 소개

3 기술 과제

4 데이터 분석 (EDA)

5 풀어야 할 과제 D0~D11: 계획과 해결

6 평가 체계: 이 점수를 믿어도 되는가

7 모델 비교

8 최종 모델과 성과

9 오분류 분석

10 배포 계획

11 한계와 리스크

출처

0. 한눈에 보기

YacssoK(약쏙)은 사진 한 장으로 알약 최대 4개의 이름과 위치를 함께 알려 주는 알약 식별 도우미다. 이 보고서는 대회 점수보다 "실제로 서비스할 수 있는가"를 기준으로, 무엇을 만들었고 무엇이 검증되었으며 출시까지 무엇이 남았는지를 정리한다.

> **세 가지 핵심**
>
> **1. 분명한 수요.** 2023년 65세 이상의 63.9%가 만성질환을 2개 이상 앓는다. 여러 약을 오래 함께 먹는 사람이 많아질수록, 약봉투 없이 남은 알약의 이름을 확인하는 일은 잦아진다.
>
> **2. 데이터로 만든 성능.** 232장으로 시작해, 같은 모델(YOLO26s)로 데이터를 바꿔 처음 보는 사진 점수(Kaggle Public)를 0.398에서 0.594로 올렸다. 팀 최고 점수는 YOLO26m의 0.60981이다.
>
> **3. 배포할 수 있는 구조.** 모델 5종의 학습 28회를 비교해, 정확도는 같은 수준이면서 가장 작고 빠른 1단계 모델(YOLO26s)을 골랐다(7장). 36분 재학습, 동작하는 웹 데모가 있고, 출시까지의 단계와 단계마다 넘어가는 기준을 정했다.

구분	내용
문제	약봉투가 없으면 낱알만 남은 약의 이름을 확인하기 어렵다. 색과 모양이 비슷한 약이 많아 눈으로 구분하기도 어렵다.
해결	사진 한 장으로 최대 4개 알약의 이름과 위치를 한 화면에 보여 준다. 확신이 낮으면 "확인 필요"를 띄운다.
검증된 성과	**Kaggle Public 0.60981 (팀 최고)** · 같은 YOLO26s로 0.398 → 0.594 · 공식 검증 0.99094, 검증 알약 156개 모두 식별 · 5-fold 재검증 0.943 ± 0.013
배포 계획	웹 데모(현재) → 서버 API → 기기 안 실행 → 식별 범위 확장 (10장)
남은 과제	처음 보는 약에 대한 일반화(Kaggle 약 0.6), 직접 찍은 사진으로 한 검증, 상용 라이선스 결정

1. 왜 YacssoK(약쏙)인가

1-1 만성질환이 늘면 먹는 약도 늘어난다

통계청 통계개발원의 『한국의 사회동향 2024』에 따르면 2023년 65세 이상 고령자의 63.9%가 만성질환을 2개 이상 앓고 있고, 1인당 평균은 2.2개다. 「2023년 노인실태조사」에서도 만성질환 수는 65~69세 평균 1.7개에서 90세 이상 3.0개로, 나이가 들수록 늘어난다.

만성질환이 늘어나기 시작하는 60대부터는 여러 약을 오랫동안 함께 먹는 사람이 많아진다. YacssoK(약쏙)이 먼저 돕고자 하는 사람은 이들, 그리고 이들의 약을 대신 챙기는 가족이다.

근거	수치	출처
65세 이상 중 만성질환 2개 이상	63.9%	『한국의 사회동향 2024』 (2023년 기준)
65세 이상 1인당 평균 만성질환 수	2.2개	『한국의 사회동향 2024』
65~69세 평균 만성질환 수	1.7개	「2023년 노인실태조사」
90세 이상 평균 만성질환 수	3.0개	「2023년 노인실태조사」

1-2 약봉투 없이 남은 알약

처방받은 약은 약봉투에 이름이 적혀 있지만, 봉투를 버리거나 약을 한 통에 옮겨 담으면 낱알만 남는다. 흰색 원형 정제처럼 색과 모양이 비슷한 약이 많아 눈으로는 구분하기 어렵고, 잘못 알아보면 같은 약을 두 번 먹거나 먹어야 할 약을 빠뜨릴 수 있다.

YacssoK(약쏙)은 이 상황에서 사진 한 장으로 알약의 위치와 이름 후보를 한 화면에 보여 준다. 그렇다면 지금은 남은 알약의 이름을 어떻게 찾고 있을까.

1-3 지금 약 이름을 찾는 방법

방법	하는 일	불편한 점
낱알식별 검색 (약학정보원, 의약품안전나라 등)	모양, 색, 각인 글자를 하나씩 골라 검색한다	알약마다 따로 검색해야 하고, 작은 각인을 읽어야 한다. 알약이 여러 개면 그만큼 반복한다.
약국·병원에 문의	약을 직접 보여 주고 확인한다	시간과 이동이 필요하다.

1-4 주요 사용자

사용자	사용 장면
여러 약을 함께 먹는 60대 이상	약봉투 없이 남은 알약, 한 통에 섞인 약을 한 번에 확인
보호자	부모님의 약을 사진으로 대신 확인
약봉투를 잃은 사람	낱알만 남은 약의 이름 확인

2. 서비스 소개

2-1 사진 한 장이 결과가 되기까지

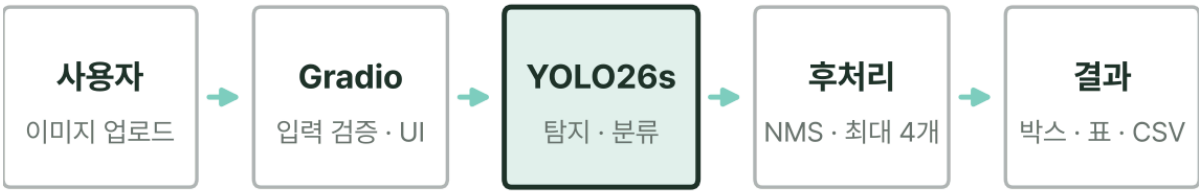


그림 2-1. 서비스 흐름. 사용자가 올린 사진을 Gradio가 검증하고, YOLO26s가 알약을 찾아 약 이름을 맞힌 뒤, 후처리에서 최대 4개로 정리해 결과를 보여 준다.

단계	하는 일	결과물
사용자	사진 업로드	알약 사진 한 장
Gradio	입력 검증, 화면(UI)	모델에 넣을 수 있는 이미지
YOLO26s	알약 탐지와 약 이름 분류	박스, 약 이름(클래스), 신뢰도
후처리	겹친 박스 정리(NMS 0.5), 이미지당 최대 4개	최종 박스
결과	박스와 이름 표시, 약 정보 표, CSV 저장	사용자 화면

> **모델은 서비스의 한 단계이며, 입력 검증과 후처리까지 함께 안정적이어야 한다.**

서비스가 받는 사진은 학습 데이터와 조건이 비슷할수록 정확하다. 학습 사진은 모두 976×1280 크기이고 한 장에 알약이 2~4개, 촬영 각도는 70·75·90도다(4장). 그래서 처음 서비스에서는 "알약 4개 이하를 위에서 찍어 주세요"처럼 학습 조건에 맞는 촬영을 안내하고, Gradio의 입력 검증 단계에서 이 조건을 확인한다.

3. 기술 과제

3-1 과제

객체 탐지(Object Detection) 문제다. 이미지 한 장에 있는 알약 최대 4개의 약 품목(클래스)과 위치(바운딩 박스)를 함께 예측한다. 학습 이미지는 232장, 평가는 Kaggle 대회의 테스트 이미지 842장으로 한다.

3-2 평가 지표 mAP75_95

예측 박스가 정답 박스와 75%, 80%, 85%, 90%, 95% 이상 겹치는지를 각각 따져 정밀도를 구하고 평균한 값이다. 박스가 "대충 그 근처"면 점수를 받지 못하고, 몇 픽셀 단위로 정확해야 한다. 이 프로젝트에서 점수를 깎은 구간도 박스 경계를 아주 정확히 맞추는 능력이었다(9장).

채점 조건	값
IoU 기준	0.75, 0.80, 0.85, 0.90, 0.95 (다섯 구간의 평균)
이미지당 최대 예측 수	4개
팀 공통 채점 함수	compute_map (신뢰도 0.001 이상, NMS 0.5, 최대 4개)

4. 데이터 분석 (EDA)

4-1 기본 수치

항목	값
학습 이미지	232장 (모두 976 × 1280)
알약 조합 폴더	114개 (같은 조합을 각도만 바꿔 여러 장 촬영)
정답 박스	763개
클래스(약 품목)	56종
이미지당 알약 수	2개 7장 · 3개 151장 · 4개 74장 (평균 3.29개)
클래스당 박스 수	최소 3 · 중앙값 9 · 최대 153
결측값	0건 (json 항목 기준. 빠진 라벨은 4-5)

항목	값
테스트 이미지	842장

4-2 클래스 분포: 한 약이 전체 박스의 20%

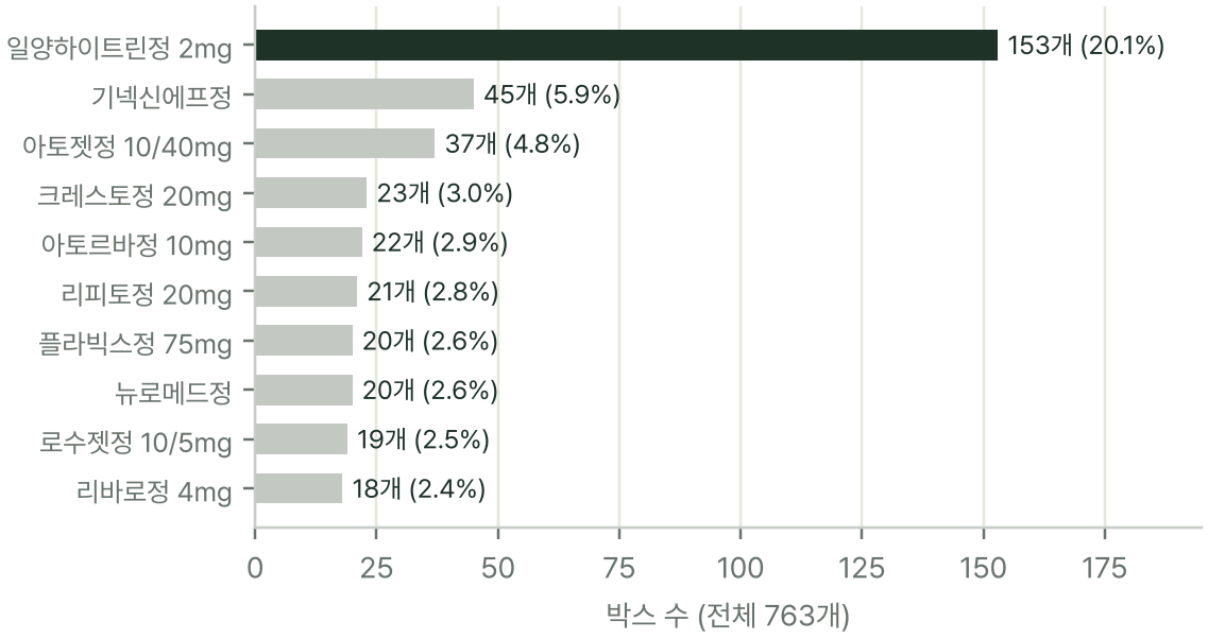


그림 4-1. 박스가 가장 많은 약 10종. 일양하이트린정 2mg 한 약이 763개 중 153개(20.05%)이고, 나머지 46종은 3~17개다.

> **해석**
>
> 모델이 많은 약에 치우치고 드문 약을 배우지 못할 위험이 크다. 드문 약은 박스가 3개뿐이라, 검증셋에 한 장만 빠져도 그 약을 한 번도 배우지 못할 수 있다.
>
> 이어진 과제: D1 오버샘플링(RFS), D2 Copy-Paste, D6 외부 데이터

4-3 알약 크기와 이미지당 알약 수

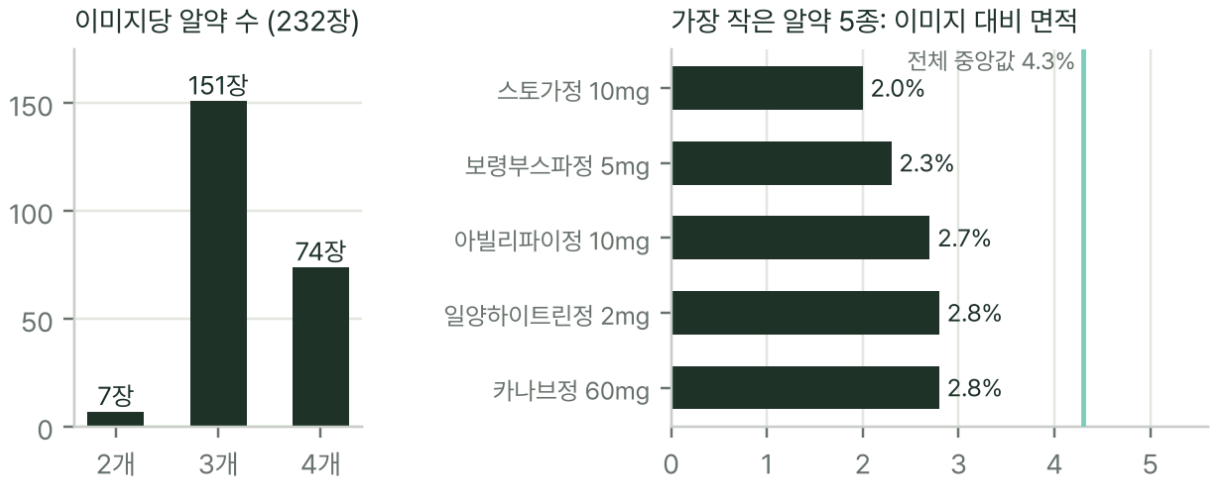


그림 4-2. 왼쪽은 이미지당 알약 수, 오른쪽은 가장 작은 알약 5종이 이미지에서 차지하는 면적. 초록 선은 전체 중앙값 4.3%다.

항목	최소	중앙값	최대
박스 가로 (px)	125	230	529
박스 세로 (px)	126	227	664
가로/세로 비율	0.40	1.00	2.11
이미지 대비 면적	1.9%	4.3%	21.8%

> **해석**
>
> 가장 작은 약(스토가정 2.0%, 보령부스파정 2.3%)은 이미지 면적의 2% 남짓이다. 입력을 640px로 줄이면 각인 글자가 몇 픽셀로 뭉개진다.
>
> 이어진 과제: D4 고해상도 학습

4-4 촬영 조건과 메타데이터

항목	발견	판단
촬영 각도	파일명과 json의 각도가 763개 모두 일치. 70·75·90도	믿을 수 있는 정보
배경색	메타데이터는 모두 "연회색"인데 실제 사진의 배경은 서로 다르다	메타데이터를 믿을 수 없어 분석에서 뺐다

항목	발견	판단
앞면·뒷면	json의 앞·뒷면 정보가 실제 사진과 맞지 않는 경우가 많고, 한쪽 면에 치우쳐 있다	면 정보 대신 각인 글자를 읽자는 아이디어로 이어짐 (D11)
조합 반복	114개 조합을 각도만 바꿔 다시 찍은 사진이 많다	사진 단위로 나누면 검증셋이 학습셋을 닮는다 (D7, 6장)

4-5 라벨 품질

사진과 정답 파일(JSON)을 하나씩 대조해 점검했다. "후보"는 오류로 확정된 것이 아니라 사진으로 다시 확인할 대상이다.

점검 항목	결과	다음 처리
사진과 JSON 짝 누락	0건	–
파일 읽기 오류 · 크기 불일치 · 중복	0건	–
정답 누락 후보 (박스가 없는 알약)	8장 (장마다 1개, 그중 4장이 일양하이트린정)	빠진 박스 추가
이미지 밖으로 나간 박스	1장	좌표 보정
크게 겹친 박스	3장 (후보)	사진으로 확인. 실제로 겹친 알약이면 그대로 둔다
약 이름(클래스) 라벨 오류	3건 (플라빅스→일양하이트린 1건, 아토젯↔트윈스타 2건)	근거 확인 후 교정표(manual_fix)에 등록
검증셋 누락 박스	4건 추가 (검증 박스 153개 → 157개)	교정표(manual_fix)에 등록

처리 원칙은 다음과 같다.

- 원본 파일은 그대로 두고, 고친 내용은 교정표에 따로 기록한다.
- 빠진 박스는 추가하고, 이미지 밖으로 나간 좌표는 보정한다.
- 약 이름 라벨은 근거를 확인한 뒤에만 고친다.
- 알약이 실제로 겹쳐 있는 정상 사례는 그대로 둔다.
- 판단하기 어려운 사진은 보류하거나 제외한다.

처음 정제한 v1에서는 문제가 있는 사진을 통째로 학습에서 빼서 232장이 219장이 되었다. 그런데 오류의 대부분은 잘못 그린 박스보다 빠진 라벨이었다. 라벨이 빠진 알약은 모델이 배경으로 배우므로 정확도에 바로 손해가 되고, 사진을 통째로 빼면 적은 데이터를 더 줄인다. 그래서 새 기준에서는 사진을 빼기보다 빠진 라벨을 채우고 틀린 라벨을 고친다.

4-6 EDA에서 과제로 (D1~D11)

EDA는 실험의 근거를 찾는 과정이고, 효과가 있는지는 같은 검증 기준의 모델 실험(5장)으로 확인한다. 아래 표는 과제마다 EDA로 무엇을 확인했고, 무엇을 찾았고, 무엇이 더 필요한지 정리한 것이다. 수치는 원본(v0) 전체 기준이며 학습 분할의 수치와 구분한다.

과제	EDA로 확인한 것	발견	아직 필요한 근거
D1 희귀 알약 반복 학습	어떤 알약의 학습 자료가 부족한가	클래스별 박스 수 최소 3개, 최대 153개	학습 분할만으로 센 클래스별 수, RFS 전후 노출 수, 희귀 클래스별 검증 성능
D2 희귀 알약 합성	잘라 붙일 알약과 빈 배경이 충분한가	클래스별로 잘라낸 알약 사진으로 재료 확인. 원본 약 번호(category_id)와 학습용 번호(label_idx)는 서로 다른 번호	합성 전후 예시와 생성 기록, 검증 사진이 합성 재료에 섞이지 않았는지
D3 안전한 이미지 증강	각도·배경·조명 변화에 대응할 필요가 있는가	각도는 70·75·90도로 파일명과 일치. 배경 메타데이터는 실제 배경색을 보증하지 않음. 반전하면 박스 좌표도 함께 바꿔야 함	실제 학습 설정(반전, 채도·명도·크기·이동, Mosaic, 회전·색상 변환 없음)과 촬영 조건별 성능
D4 고해상도 학습	입력을 줄이면 작은 알약의 특징이 사라지는가	640 입력에서 알약 짧은 변이 대부분 70~120px(50px대도 있음), 1024 입력에서는 130~200px. 파일 읽기 오류·크기 불일치·중복은 0	같은 조건에서 해상도만 바꾼 비교, 각인이 실제로 읽히는지 사진 확인
D5 잘못된 데이터 제거	학습을 방해하는 파일·정답 오류가 있는가	사진과 JSON의 짝 누락 0. 정답 누락 후보 8장, 경계 밖 박스 1장, 크게 겹친 후보 3장 (오류 확정이 아니라 검토 대상)	전체 검수 범위, 제외 목록과 사유, 제외 전후 클래스별 수
D6 외부 데이터	부족한 약과 촬영 상황을 외부 데이터가 채우는가	AI Hub 라벨 6,011개: 약 41종 중 21종은 기존과 같고 20종은 새 약. 각도(70·75·90도)와 크기(976×1280)가 같음. 박스 20개 형식 오류. 약 번호는 drug_N 기준으로 맞춰야 함	대회 허용 범위, 원본 검증·테스트와의 중복 확인, 클래스 대응표 검증

과제	EDA로 확인한 것	발견	아직 필요한 근거
D7 여러 분할로 재검증	나누는 방식에 따라 점수가 달라지는가	같은 조합을 각도만 바꿔 찍은 사진이 많음 (조합 114개)	fold별 클래스·사진 수, 조합 중복 여부, 평균과 표준편차
D8 모델 예측을 재학습에 활용	정답 없는 사진을 임시 정답으로 쓸 수 있는가	테스트 842장 읽기 오류 0. 정답이 없어 정확도는 계산할 수 없음	대회 규칙상 허용 여부, 독립 검증셋, 신뢰도 기준과 사람 검수
D9 더 강한 증강 (MixUp)	D3보다 강한 변화가 도움이 되는가, 식별 단서를 망가뜨리는가	MixUp 결과나 강도별 비교는 아직 없음	같은 사진에 약한·강한 증강을 적용한 예시, 각인·색 구분 확인, 모델 실험
D10 틀린 라벨 직접 수정	모델 오류처럼 보이는 사례가 실제로는 정답 오류인가	종류 라벨 오류 3건(플라빅스→일양하이트린 1건, 아토젯↔트윈스타 2건)과 누락 박스 4건이 교정표에 등록됨. 검증 박스 153개 → 157개	교정 근거 기록, 나머지 누락 후보의 처리
D11 알약 글자 읽기 (OCR)	색과 모양이 비슷한 약을 각인으로 구분할 수 있는가	OCR 분석은 아직 없음. 모델 비교와 혼동 확인 뒤 진행 예정	실제로 헷갈리는 약 쌍, 각인 확대 사진과 글자 정답, OCR 정확도

D0(원본 기준)은 EDA 대상이 아니라 비교 기준이라 표에서 뺐다.

> **EDA 점검이 평가 점수에 준 영향**
>
> 교정표(manual_fix)를 반영한 검증 정답으로 이미 저장된 예측을 다시 채점하자, D1(RFS)은 0.96903 → 0.98341, D1+증강은 0.93817 → 0.95234로 올랐다. 모델을 다시 학습한 것이 아니라 정답만 고친 결과다. 라벨 오류가 점수를 깎고 있었다는 뜻이다.
>
> manual_fix만 반영했고 manual_drop은 포함하지 않았다. 교정 전후 점수는 섞어 비교하지 않는다.

5. 풀어야 할 과제 D0~D11: 계획과 해결

데이터 개선은 한 번에 섞지 않고 단계별로 비교했다. 원본 기준(D0)을 먼저 정하고, 매번 한 가지만 바꿔 같은 채점 함수로 비교했다. 재실행만으로도 점수가 0.019 흔들리므로(6장), 이보다 작은 차이는 효과로 보지 않았다.

5-1 과제별 계획과 결과

과제	왜 (EDA 근거)	계획	결과
D0 원본 기준	비교의 출발점	원본 그대로 학습	검증 8s 0.933 · 11s 0.947 · 26s 0.949 (같은 Mac 환경)
D1 오버 샘플링	한 약이 20%	드문 약이 든 사진을 더 자주 학습 (RFS 0.1)	검증은 4개 조합 모두 상승(+0.004~+0.028). 원본 데이터에서는 Kaggle이 오르지 않음(0.380~0.398)
D2 Copy-Paste	드문 약은 박스 3개	드문 약을 잘라 다른 사진에 붙여 합성	단독으로는 −0.006~+0.011로 방향이 갈림. RFS와 함께 최종 구성(d12)에 포함
D3 이미지 증강	적은 데이터	반전, 밝기 등	26m −0.018로 하락. 각인 방향과 색 경계가 바뀌어 다른 약처럼 보일 수 있음. 제외
D4 고해상도	작은 알약	입력 1024px	Kaggle 0.368(최하위). batch와 재시작이 함께 바뀌어 원인 분리 실패
D5 라벨 정제 강화	라벨 누락·오류	사람이 한 장씩 확인해 제외	232장 → 219장. 점수 차이는 흔들림 안
D6 외부 데이터	처음 보는 약·조합	AI Hub 경구약제 조합 데이터 (대회 원본과 겹치는 데이터는 사용 금지)	**같은 26s로 Kaggle 0.418 → 0.544 → 0.594.** 가장 큰 개선
D7 K-fold	같은 조합 반복	5-fold로 검증셋을 바꿔 가며 다시 채점	원본(v0)에서는 fold 편차 0.033~0.063으로 seed보다 3~8배 컸다. 최종 설정(v3allcp)은 평균 0.943 ± 0.013 (5-3)
D8 테스트 이미지 라벨링·증강	테스트 분포	테스트와 비슷한 사진 보강	진행하지 않음
D9 MixUp	적은 데이터	두 사진을 섞어 증강	최종 구성에는 넣지 않음
D10 틀린 라벨 수정	박스·클래스 오류	클래스와 박스를 직접 수정	종류 라벨 3건과 누락 박스 4건을 고쳐 v3allc에 반영. 교정 정답으로 다시 채점하면 0.969 → 0.983 (4-6)

과제	왜 (EDA 근거)	계획	결과
D11 각인 OCR	닮은 약	각인 글자로 약 이름 다시 고르기	0.106으로 급락. OCR을 꺼도 같은 점수라 클래스 번호 매핑 문제로 보임. 제외

5-2 데이터 버전 (V)

V는 데이터 버전(Version)이다. 원본은 그대로 두고, 과제(D)를 적용한 결과를 새 버전으로 저장했다. D가 "무엇을 할지"라면 V는 "그 결과로 만든 데이터"다. 한 실험에서는 한 가지만 바꾸기 때문에 점수가 달라지면 어느 버전 차이 때문인지 알 수 있고, 같은 버전으로 언제든 다시 학습할 수 있다.

가장 많이 쓴 v3allc 위에서는 데이터는 두고 학습 방법만 바꿔 실험했고, 실험 이름 끝에 d1(RFS), d2(Copy-Paste), d3(안전 증강), d12(RFS + Copy-Paste)를 붙여 구분했다.

버전	만든 방법	적용 과제	클래스 수
v0	대회 원본 (232장)	D0	56
v1	라벨 정제: 문제가 있는 사진을 제외 (232장 → 219장)	D5	56
v2	Copy-Paste 합성: 박스 10개 미만인 약을 잘라 다른 사진에 붙여 보강	D2	56
v3	AI Hub 조합 데이터 중 기존 56종 약만 든 사진을 추가	D6	56
v3all	AI Hub의 새 약까지 클래스로 추가	D6	76 (기존 56 + 새 약 20)
v3allc	v3all 수정판: 라벨 정제 강화(D5)와 틀린 라벨 수정(D10)을 더함	D5, D10	76
v3allcp	v3allc에 Copy-Paste 합성을 더한 것 (d12 실험에 사용)	D2	76

5-3 해결의 핵심: 모델이 아니라 데이터

Kaggle 점수를 움직인 것은 모델 교체가 아니라 데이터였다. 원본부터 Copy-Paste(v0~v2)까지는 모델(8s, 11s, 26s)과 기법(RFS, Copy-Paste)을 바꿔도 Kaggle이 0.38~0.41에 머물렀다. AI Hub 데이터로 처음 보는 약과 조합을 보여 주자 같은 YOLO26s가 0.418, 새 클래스를 더한 v3all에서 0.544, 라벨을 고친 v3allc에서 0.594까지 올랐다.

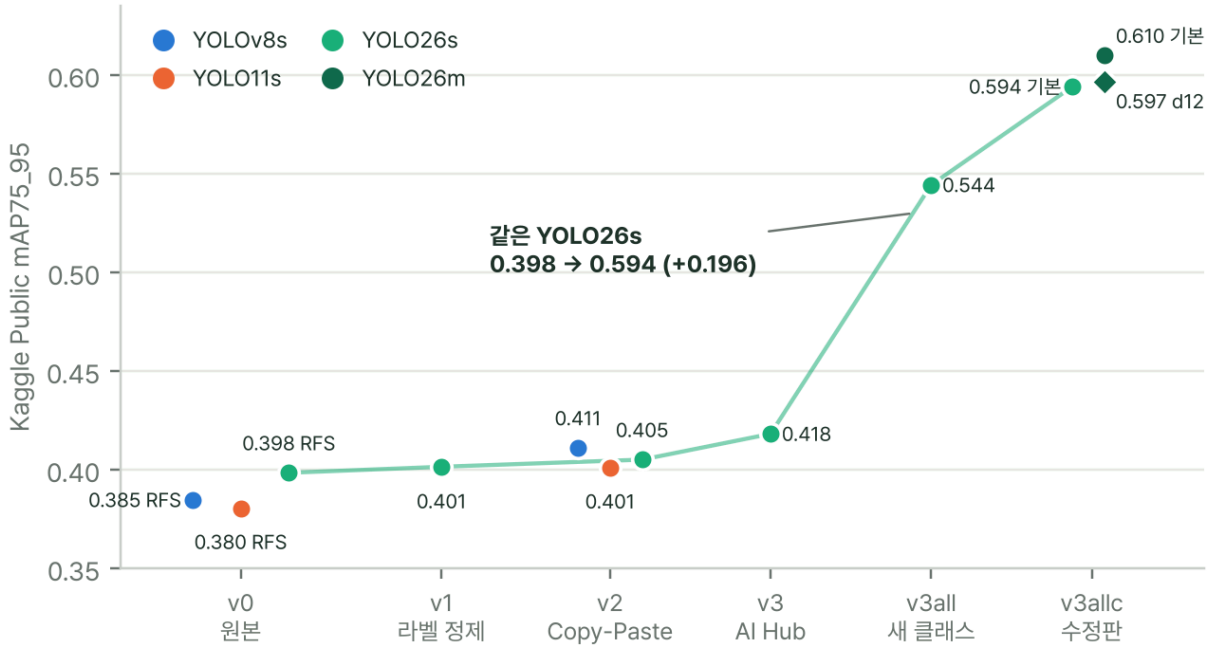


그림 5-1. 데이터 버전별 Kaggle Public 점수. 초록 선은 같은 YOLO26s의 변화다. batch 8로 학습한 실험은 뺐다. v3allc의 기본 두 점(26s 0.594, 26m 0.610)은 30 epoch 학습이다.

> **외부 데이터는 효과보다 매핑부터 검증한다**
>
> 새 클래스를 더한 v3all에서 0~75번 클래스 중 **59개가 서로 다른 약**을 가리키고 있었다. 데이터를 만드는 단계와 클래스표를 만드는 단계가 서로 다른 순서를 썼기 때문이다.
>
> 번호가 아니라 약 이름을 기준으로 매핑을 다시 맞추고, AI Hub에만 있는 6개 클래스는 제출에서 뺐다. 이 문제를 놓쳤다면 59개 클래스가 사실상 0점이 되었다.

최종 데이터에서 학습 방법 고르기

데이터를 v3allc로 정한 뒤에는 데이터는 그대로 두고 학습 방법만 바꿔 비교했다. 같은 Windows GPU, batch 16, 640px, 50 epoch 조건에서 네 가지 방법(d1 RFS, d2 Copy-Paste, d3 안전 증강, d12 RFS + Copy-Paste)을 YOLO26s와 YOLO26m 두 크기로 학습했다.

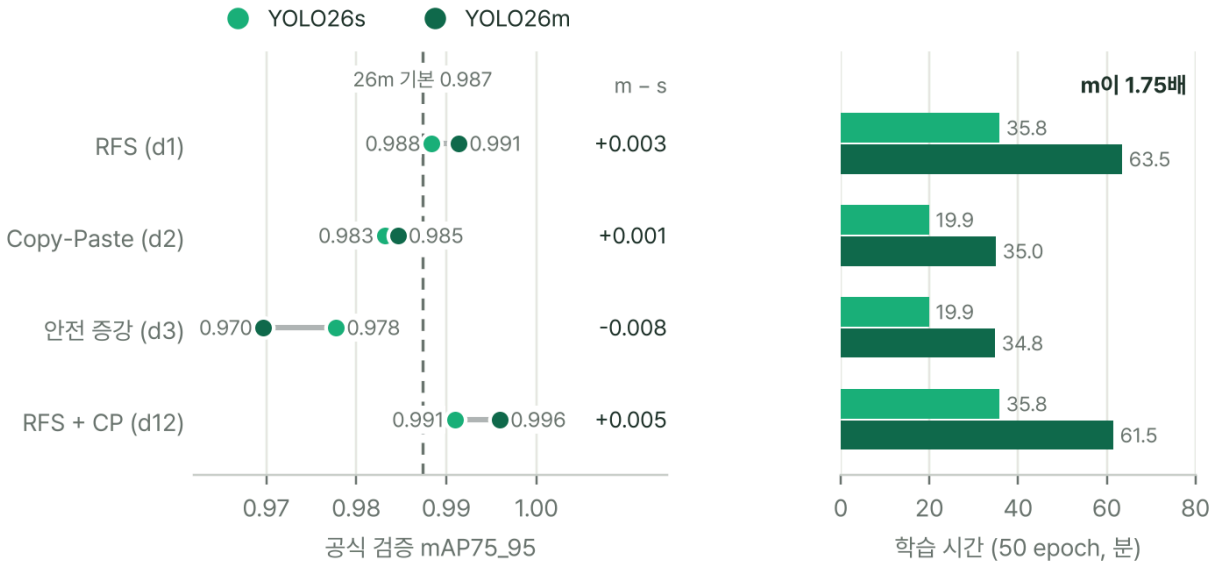


그림 5-2. 최종 데이터(v3allc)에서 학습 방법별 공식 검증 점수(왼쪽)와 학습 시간(오른쪽). 점선은 YOLO26m 기본 설정의 점수다.

방법	YOLO26s	YOLO26m	26m 기본 대비	학습 시간 s / m
기본	–	0.98737	–	– / 35.0분
RFS (d1)	0.98833	0.99137	+0.004	35.8 / 63.5분
Copy-Paste (d2)	0.98315	0.98461	−0.003	19.9 / 35.0분
안전 증강 (d3)	0.97775	0.96966	−0.018	19.9 / 34.8분
RFS + Copy-Paste (d12)	**0.99094**	**0.99593**	**+0.009**	35.8 / 61.5분

- 두 크기 모두 **RFS + Copy-Paste(d12)가 가장 높고, 안전 증강(d3)이 가장 낮다.** 어떤 방법이 좋은지는 모델 크기와 상관없이 같았다.
- 안전 증강은 26m에서 기본보다 0.018 떨어졌다. 반전과 밝기 변화가 각인 방향과 색 경계를 바꿔 다른 약처럼 보이게 한다는 판단(5-1 D3)과 맞는다.
- d12의 개선(26m 기본 대비 +0.009)은 재실행 흔들림(0.019)보다 작고, 공식 검증셋은 점수가 부풀려져 있다(6-2). 그래서 이 차이만으로 정하지 않고 아래 5-fold로 다시 확인했다.

- 같은 방법에서 두 크기의 차이는 평균 +0.0004로 흔들림 안이고, 학습 시간은 26m이 1.75배다. 서비스 모델을 YOLO26s로 정한 근거다(10-2).

> **검증 최고가 Kaggle 최고는 아니다**
>
> YOLO26m에서 RFS와 Copy-Paste를 함께 쓴 d12는 검증 0.99593으로 가장 높았지만, Kaggle은 0.59671로 기본 설정(0.60981)보다 낮았다. 학습 사진을 반복하고 합성하는 기법은 학습과 닮은 검증셋에서 더 좋아 보인다. 그래서 최종 판단은 처음 보는 사진 점수로 한다(6장).

최종 설정 재검증: 5-fold

최종 설정은 YOLO26s에 v3allcp(v3allc + Copy-Paste 합성)와 RFS 0.1을 쓰는 d12다. 공식 검증셋은 47장 중 34장이 학습셋과 같은 조합이라(6-2), 이 설정을 5-fold로 다시 검증했다. 학습셋을 다섯 묶음으로 나눠 매번 한 묶음으로 검증하고 나머지로 학습했다. Copy-Paste 합성은 fold마다 그 fold의 학습 사진으로만 새로 만들어, 검증 사진이 합성 재료로 섞이지 않게 했다. 학습 조건은 단일 모델과 같다(batch 16, 640px, 50 epoch, seed 42).

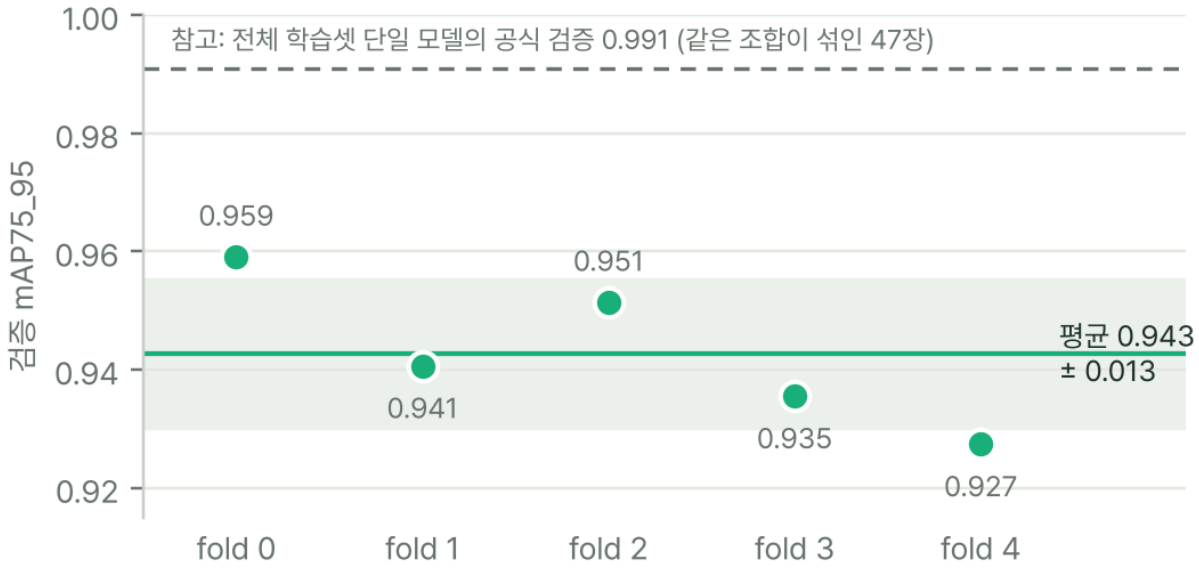


그림 5-3. fold별 검증 점수. 초록 선은 평균, 띠는 ± 표준편차다. 점선은 전체 학습셋으로 학습한 같은 설정의 단일 모델이 공식 검증셋에서 받은 점수로, 검증 사진이 달라 참고로만 표시했다.

실행	검증 mAP75_95	IoU 0.75 (참고)	학습 시간
fold 0	0.95901	0.96875	39.4분

실행	검증 mAP75_95	IoU 0.75 (참고)	학습 시간
fold 1	0.94061	0.94634	30.5분
fold 2	0.95134	0.97616	33.8분
fold 3	0.93549	0.95163	31.7분
fold 4	0.92736	0.93497	29.4분
5-fold 평균 ± 표준편차	**0.943 ± 0.013**	0.956 ± 0.017	32.9분
단일 모델 (전체 학습셋, 공식 검증 47장)	0.99094	1.00000	35.8분

- **어떤 사진으로 검증해도 점수가 비슷하다.** 다섯 fold가 0.927~0.959에 있고 표준편차는 0.013으로, 재실행 흔들림(0.019)보다 작다.
- **공식 검증 점수는 부풀려져 있다.** fold 평균(0.943)은 단일 모델의 공식 검증(0.991)보다 약 0.05 낮다. fold 모델이 학습셋의 약 80%만 학습했다는 차이도 있지만, 학습과 같은 조합이 섞인 공식 검증셋이 실제보다 높게 나온다는 6-2의 설명과 같은 방향이다.
- **재학습 시간이 일정하다.** fold마다 29~39분, 평균 33분으로 단일 모델(35.8분)과 비슷하다.

> **5-fold 앙상블**
>
> fold 모델 다섯 개의 예측을 모아 클래스별 NMS(IoU 0.5, 가중치 모두 1.0, 이미지당 최대 4개, 신뢰도 0.001)로 합치는 설정도 준비했다.
>
> 각 fold 모델이 다른 fold의 검증 사진을 이미 학습했기 때문에 앙상블 점수는 검증셋으로 공정하게 잴 수 없다. 그래서 제출은 단일 모델을 기본으로 하고, 앙상블은 Kaggle Public 점수로만 비교한다.

5-4 한계와 더 생각할 지점

- D4 고해상도는 해상도 말고도 batch(16 → 8)와 학습 재시작이 함께 바뀌어 효과를 판단하지 못했다. 같은 조건으로 다시 실험해야 한다.
- 빠진 라벨은 "빼기"보다 "추가하기"가 맞는 처방이다. 232장 중 13장(5.6%)을 버린 것은 적은 데이터에서 손해가 크다.

- D8 테스트 이미지 라벨링·증강은 실행하지 않았다.

6. 평가 체계: 이 점수를 믿어도 되는가

데이터가 적을수록 점수는 쉽게 흔들리고, 흔들리는 점수로 고른 모델은 처음 보는 사진에서 실패한다. 그래서 점수를 올리기 전에, 그 점수를 믿어도 되는지부터 확인했다.

6-1 같은 실험을 두 번 돌려도 0.019 흔들린다

실행	검증 mAP75_95
YOLO26s 기준 실행	0.94894
같은 설정으로 재실행	0.92945
차이	**0.0195**

모델, 데이터, seed가 같아도 Mac GPU(mps)에서는 epoch 4부터 결과가 갈라졌다. 그래서 이 흔들림보다 작은 차이는 "효과 있음"으로 보지 않는다.

6-2 공식 검증셋에 학습 사진과 같은 조합이 섞여 있다

공식 검증 47장 중 **34장이 학습셋과 같은 알약 조합**이었다. 각도만 바꾼 사진이 양쪽에 있어, 모델이 외운 사진과 거의 같은 사진으로 시험을 보는 셈이다. 공식 검증 점수가 0.93~0.99로 높게 나오는 이유다. 최종 설정을 5-fold로 다시 재면 평균 0.943으로, 공식 검증(0.991)보다 약 0.05 낮다(5-3).

6-3 검증 점수로는 Kaggle 점수를 예측할 수 없었다

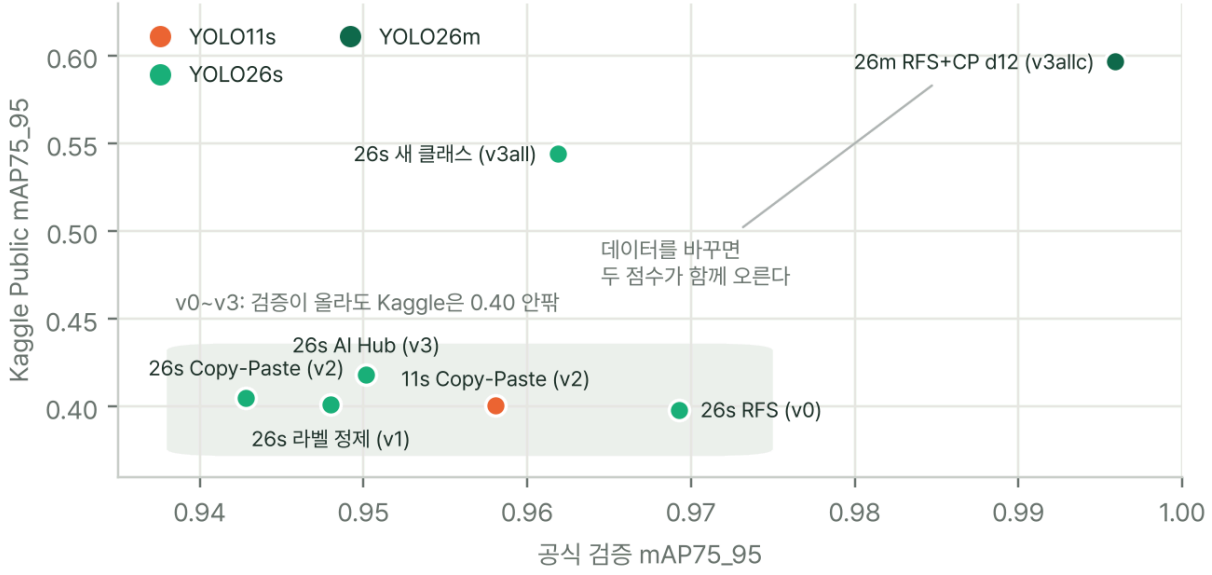


그림 6-1. 같은 실험의 공식 검증 점수와 Kaggle Public 점수. 검증과 Kaggle이 모두 있는 실험만 표시했다.

실험	데이터	공식 검증	Kaggle Public
YOLO26s RFS	v0	0.96928	0.39833
YOLO11s Copy-Paste	v2	0.95807	0.40090
YOLO26s AI Hub	v3	0.95016	0.41806
YOLO26s 라벨 정제	v1	0.94801 (43장)	0.40138
YOLO26s Copy-Paste	v2	0.94284	0.40500
YOLO26s 새 클래스	v3all	0.96188	0.54422
YOLO26m RFS + Copy-Paste (d12)	v3allc	**0.99593**	**0.59671**

v0~v3에서는 검증 점수가 0.94~0.97로 달라도 Kaggle은 0.40 안팎에 머물렀다. 검증에서 1위였던 RFS가 Kaggle에서는 하위권이었다. 학습 사진을 반복하는 RFS가 학습과 닮은 검증 사진에서만 좋아 보였기 때문이다. 데이터를 바꾼 v3all과 v3allc에서야 두 점수가 함께 올랐다.

6-4 점수를 흔드는 것은 seed보다 검증셋 구성

모델	seed만 바꿈	fold를 바꿈	배수
YOLO11s	0.020	0.063	약 3배

모델	seed만 바꿈	fold를 바꿈	배수
YOLOv8m	0.007	0.055	약 8배

표의 값은 표준편차다. 어떤 사진이 검증셋에 들어가느냐가 seed보다 점수를 훨씬 크게 흔든다.

6-5 그래서 세운 판정 규칙

1. 공식 검증 점수는 같은 환경 안에서 "기준 대비 개선 폭"으로만 읽는다.
2. 개선 폭이 재실행 흔들림(0.019)보다 커야 효과로 인정한다.
3. 실험 환경(PC, batch, 해상도)이 다르면 점수를 나란히 놓고 우열을 가리지 않는다.
4. 최종 판단은 처음 보는 사진 점수(Kaggle)로 한다.
5. 모든 점수는 같은 채점 함수(compute_map: 신뢰도 0.001, NMS 0.5, 최대 4개, IoU 0.75~0.95)로 낸다.

7. 모델 비교

서비스에 올릴 모델은 "가장 높은 점수"가 아니라 "정확도에서 뒤지지 않으면서 가장 싸고 빠르게 학습하고 배포할 수 있는 모델"로 골랐다. 팀이 학습한 결과 파일을 모두 모아 모델별로 다시 분석했고, 후보는 구조(Faster R-CNN과 YOLO), 세대(YOLOv8s, YOLO11s, YOLO26s), 크기(YOLO26s와 YOLO26m) 순서로 좁혔다.

7-1 비교 기준과 실험 환경

같은 모델이라도 batch, 학습 기기, 데이터가 다르면 점수가 달라진다. 이런 실행끼리 점수를 나란히 놓으면 차이가 모델 때문인지 환경 때문인지 가를 수 없다. 그래서 받은 실행 28개를 환경이 같은 것끼리 네 묶음으로 나누고, 묶음 안에서만 비교했다.

묶음	조건	데이터	모델	실행 수
A (7-2)	Colab GPU · 20 epoch	v1 (학습 185장 / 검증 47장)	Faster R-CNN, YOLOv8s	2
B (7-3)	Mac (mps) · batch 16 · 640px · 50 epoch	v0, v1, v2, v3, v3all	YOLOv8s, YOLO11s, YOLO26s	12
C (7-4)	Windows GPU · batch 16 · 640px · 50 epoch	v3allc, v3allcp	YOLO26s, YOLO26m	9
D (5-3)	batch 16 · 640px · 50 epoch · 5-fold	v3allcp (fold마다 합성)	YOLO26s × 5	5

batch 8로 학습한 실행 5개(별도 PC의 YOLOv8m, YOLO11m, YOLO12m 기본과 YOLO11m RFS, 1024px로 학습한 YOLO26s)는 조건이 달라 비교에서 뺐다. 차이는 재실행 흔들림 0.019보다 커야 효과로 보고(6-5), 정확도가 같은 수준이면 학습 시간과 배포 구조로 고른다. 최종 판단은 처음 보는 사진 점수(Kaggle Public)로 한다.

7-2 2단계 모델(Faster R-CNN)과 YOLO (묶음 A)

Faster R-CNN은 후보 영역을 먼저 찾고(1단계) 영역마다 이름과 박스를 다시 맞추는(2단계) 구조라, 박스를 정밀하게 맞추는 데 강하다고 알려져 있다. 대회 지표가 박스 정밀도를 따지는 만큼 꼭 확인해야 할 후보였다. 그래서 YOLOv8s와 같은 데이터(v1 분할), 같은 학습 횟수(20 epoch)로 비교했다.

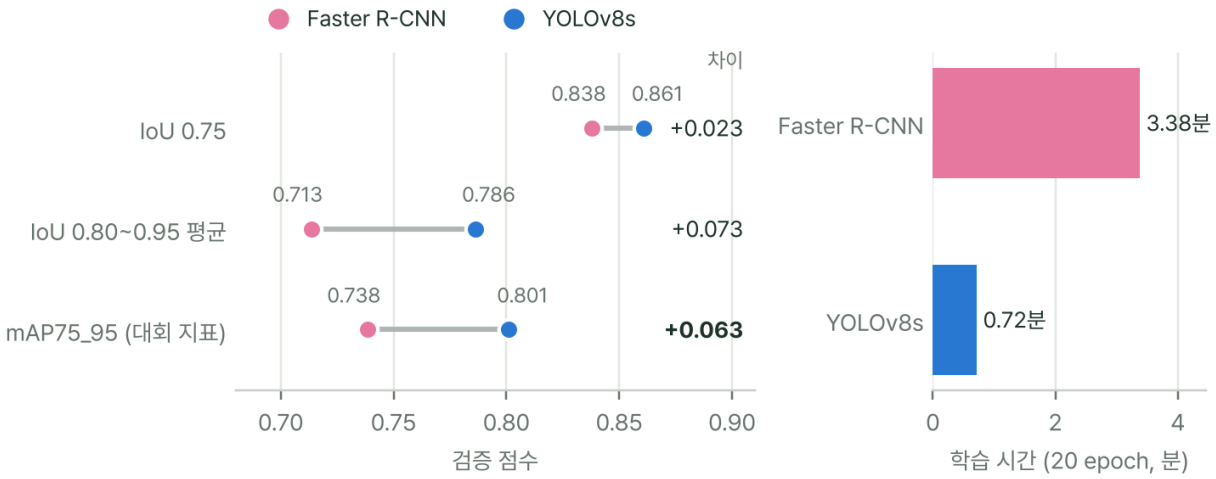


그림 7-1. 같은 v1 분할에서 20 epoch씩 학습한 결과. 왼쪽은 IoU 기준별 검증 점수이고, IoU 0.80~0.95 평균은 mAP75_95(다섯 기준의 평균)와 IoU 0.75 점수로 역산했다. 오른쪽은 학습 시간이다.

항목	Faster R-CNN	YOLOv8s	차이
mAP75_95 (대회 지표)	0.73839	**0.80109**	**+0.063**
IoU 0.75 (참고)	0.83801	0.86084	+0.023
IoU 0.80~0.95 평균 (참고, 역산)	0.71348	0.78615	+0.073
학습 시간 (20 epoch)	3.38분	**0.72분**	4.7배 빠름
구조	2단계	1단계	–

- **정확도:** 같은 조건에서 YOLO가 0.063 높다. 재실행 흔들림(0.019)의 세 배가 넘는다.
- **박스 정밀도:** 느슨한 IoU 0.75에서는 차이가 0.023이지만, 엄격한 IoU 0.80~0.95에서는 0.073으로 세 배 넘게 벌어진다. 대회 지표와 서비스가 요구하는 것이 바로 이 엄격한 박스다.
- **비용:** 학습이 4.7배 빠르다. 같은 시간에 실험을 4~5번 더 돌릴 수 있고, 새 약을 추가한 뒤 다시 학습하는 시간도 그만큼 짧다.

두 모델 모두 20 epoch만 학습해 점수 수준이 다른 묶음보다 낮으므로, 다른 묶음의 점수와는 비교하지 않는다. 비교한 YOLO는 YOLOv8s지만 7-3에서 YOLOv8s와 YOLO26s의 정확도가 같은 수준이라 결론은 같다.

> **결론:** Faster R-CNN은 정확도, 학습 비용, 배포 구조 모두에서 불리해 후보에서 뺐다.

7-3 YOLO 세대 비교 (묶음 B · Mac · batch 16)

같은 Mac, 같은 batch 16, 같은 640px·50 epoch 조건에서 세 세대의 작은(s) 모델을 비교했다. 기본(v0)은 세 모델이 모두 있고, RFS와 Copy-Paste는 두 모델씩 있다.

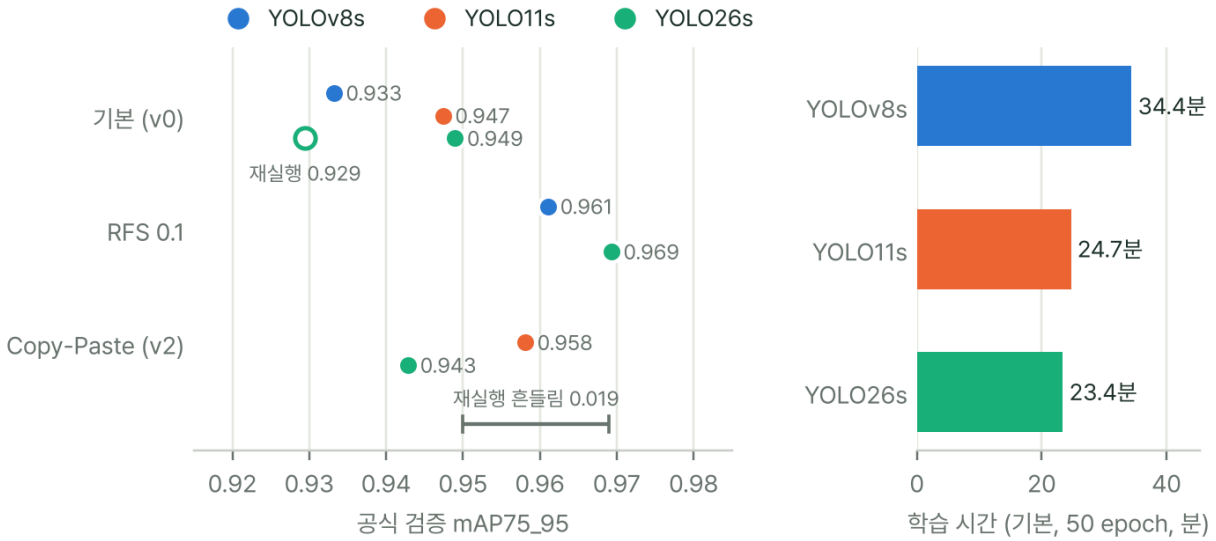


그림 7-2. 왼쪽은 실험별 공식 검증 점수이고, 빈 원은 YOLO26s 기본을 같은 설정으로 다시 돌린 결과다. 아래 막대는 재실행 흔들림(0.019)의 길이다. 오른쪽은 기본 실험의 학습 시간이다.

실험	YOLOv8s	YOLO11s	YOLO26s	읽을 것
기본 (v0)	0.93319	0.94739	**0.94894**	26s와 8s 차이 0.016. 흔들림 안
RFS 0.1 (v0)	0.96106	–	**0.96928**	26s가 0.008 높지만 흔들림 안
Copy-Paste (v2)	–	**0.95807**	0.94284	11s가 0.015 높지만 흔들림 안
Kaggle (Copy-Paste)	–	0.40090	**0.40500**	차이 0.004
학습 시간 (기본)	34.4분	24.7분	**23.4분**	26s가 가장 빠름. 8s는 1.47배

- **다른 모델 · 같은 실험:** 세 실험 모두 모델 사이 차이가 0.019보다 작고, 실험마다 앞서는 모델도 바뀐다. 이 조건에서 세대에 따른 정확도 차이는 확인되지 않았다.
- **같은 모델 · 다른 실험:** 흔들림을 확실히 넘은 것은 YOLOv8s의 RFS(+0.028)와 라벨 정제(v1, +0.042)다. YOLO26s의 RFS(+0.020)는 경계이고, Copy-Paste(−0.006), AI Hub(v3, +0.001), 새 클래스(v3all, +0.013)는 흔들림 안이다. YOLO26s의 라벨 정제 실행은 검증 사진 수가 달라(43장) 나란히 비교하지 않는다.

- **검증과 Kaggle:** YOLO26s는 검증이 0.94~0.97에 머무는 동안 Kaggle이 0.398(RFS)에서 0.544(v3all)로 올랐다. 모델보다 데이터가 처음 보는 사진 점수를 움직였다(5-3, 6-3).
- **학습 비용:** 같은 모델에서 RFS와 Copy-Paste는 학습 시간이 1.5~2.1배, v3all은 데이터가 늘어 226분(기본의 약 10배)이 걸렸다. 세대 사이에서는 YOLO26s가 가장 빠르고, NMS 없이 최종 박스를 내도록 설계되어 배포할 때 후처리를 줄일 여지가 있다.

> **결론:** 정확도는 세 세대가 같은 수준이고, 학습이 가장 빠른 YOLO26을 골랐다.

7-4 모델 크기 비교 (묶음 C · Windows GPU · batch 16)

최종 데이터(v3allc)에서 같은 학습 방법 네 가지를 두 크기(YOLO26s, YOLO26m)로 학습했다. 방법별 점수는 그림 5-2에 있고, 여기서는 같은 방법끼리 두 크기의 차이와 학습 비용을 본다.

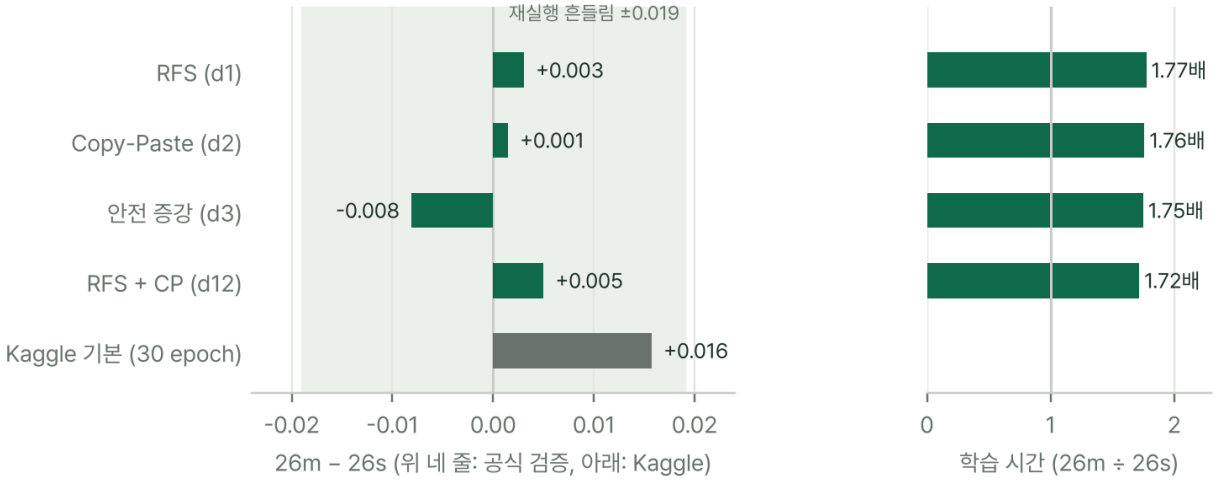


그림 7-3. 왼쪽은 같은 방법에서 26m과 26s의 점수 차이(회색 띠는 재실행 흔들림 ±0.019, 맨 아래는 Kaggle Public), 오른쪽은 26m의 학습 시간이 26s의 몇 배인지다.

방법	YOLO26s	YOLO26m	차이 (m − s)	학습 시간 s / m
RFS (d1)	0.98833	0.99137	+0.003	35.8 / 63.5분
Copy-Paste (d2)	0.98315	0.98461	+0.001	19.9 / 35.0분
안전 증강 (d3)	0.97775	0.96966	−0.008	19.9 / 34.8분
RFS + Copy-Paste (d12)	**0.99094**	**0.99593**	+0.005	35.8 / 61.5분
네 쌍 평균			**+0.0004**	**m이 1.75배**

Kaggle Public (처음 보는 사진)	YOLO26s	YOLO26m	차이 (m − s)
v3allc 기본 (30 epoch)	0.59408	**0.60981**	+0.016
RFS + Copy-Paste (d12, 50 epoch)	미제출	0.59671	–

- 26m이 네 쌍 중 세 쌍에서 조금 높지만 평균 차이는 +0.0004로 재실행 흔들림(0.019)의 50분의 1이다. Kaggle에서도 기본 설정의 차이는 0.016으로 흔들림보다 작다.
- 학습 시간은 네 쌍 모두 26m이 1.72~1.77배로 일정하게 길고, 모델이 커서 기기에서 돌리기도 더 어렵다.
- 두 크기 모두 RFS + Copy-Paste가 가장 높고 안전 증강이 가장 낮다. 어떤 방법이 좋은지는 모델 크기와 상관없이 같았다.
- Kaggle 팀 최고 점수(0.60981)는 26m이다. 26m은 버리지 않고, "확인 필요" 사진만 서버에서 다시 보는 큰 모델 후보로 보관한다.

> **결론:** 큰 모델의 정확도 이점은 흔들림 안이고 학습 비용은 1.75배라, 서비스 모델은 YOLO26s로 정했다.

7-5 왜 YOLO26s인가: 선택 근거

모델은 한 번에 고르지 않고, 질문 여섯 개에 차례로 답하며 좁혔다. 각 답은 위 묶음의 실행에서 나왔고, 외부 데이터(AI Hub)를 넣은 뒤에도 같은 결론인지 최종 데이터에서 다시 확인했다.

질문	근거 실행	결과	결정
① 1단계냐 2단계냐	묶음 A · 2개	YOLO가 +0.063, 엄격한 IoU 0.80~0.95에서는 +0.073. 학습은 4.7배 빠름	YOLO
② 어느 세대인가	묶음 B · 12개	8s · 11s · 26s의 차이가 세 실험 모두 흔들림(0.019) 안. 기본 학습은 26s가 23.4분으로 가장 빠름	YOLO26
③ 외부 데이터는 효과가 있나	묶음 B v3 · v3all, 묶음 C v3allc	같은 YOLO26s로 Kaggle 0.398(원본 RFS) → 0.418(v3 AI Hub) → 0.544(v3all 새 클래스) → 0.594(v3allc 라벨 수정). 모델 교체(0.38~0.41)보다 훨씬 큰 폭	AI Hub를 더한 v3allc를 최종 데이터로
④ 큰 모델이 필요한가	묶음 C · 9개 + Kaggle 2개	외부 데이터를 넣은 최종 데이터에서 26m − 26s 평균 +0.0004, Kaggle 차이 0.016. 모두 흔들림 안. 26m 학습은 1.75배	YOLO26s (26m은 서버용 보관)
⑤ 어떤 학습 방법인가	묶음 C d1 · d2 · d3 · d12	두 크기 모두 RFS + Copy-Paste(d12)가 가장 높고 안전 증강(d3)이 가장 낮음	v3allcp + RFS 0.1 (d12)

질문	근거 실행	결과	결정
⑥ 믿을 수 있나	묶음 D · 5개	5-fold 0.943 ± 0.013. 표준편차가 흔들림보다 작고, fold마다 재학습 29~39분	**확정**

받은 실행 28개를 모델별로 모으면 다음과 같다. 검증 점수는 묶음마다 조건이 달라 같은 모델 안에서도 서로 비교하지 않고, 어떤 실행이 가장 높았는지만 적었다.

모델	실행 수 (묶음)	검증 최고 (실행)	Kaggle 최고	학습 시간	판단
Faster R-CNN	1 (A)	0.73839 (v1, 20 epoch)	–	3.38분 (20 epoch)	제외. 같은 조건의 YOLOv8s보다 −0.063, 4.7배 느림
YOLOv8s	4 (A 1, B 3)	0.97547 (v1 라벨 정제)	–	22.5~52.2분	제외. 26s와 같은 수준, 학습 1.47배
YOLO11s	2 (B)	0.95807 (v2 Copy-Paste)	0.40090	24.7~43.1분	제외. 26s와 흔들림 안
YOLO26s	16 (B 7, C 4, D 5)	**0.99094** (v3allcp d12)	0.59408 (v3allc 기본)	19.9~48.5분 (v3all 226분)	**서비스 모델.** 5-fold 0.943 ± 0.013 (5-3)
YOLO26m	5 (C)	0.99593 (v3allcp d12)	**0.60981** (v3allc 기본)	34.8~63.5분	보관. Kaggle 팀 최고, 서버용 큰 모델 후보

> **서비스 모델은 YOLO26s(v3allcp, RFS 0.1, 50 epoch)다.** 구조 비교에서는 YOLO가 정확도와 비용 모두 앞섰고, 세대와 크기 비교에서는 정확도 차이가 흔들림 안이라 학습이 가장 빠르고 작은 모델을 골랐다. 처음 보는 사진 점수를 올린 것은 모델이 아니라 외부 데이터였고(③), 그 데이터 위에서도 같은 결론이었다(④). 고른 설정은 5-fold에서도 0.943 ± 0.013으로 안정적이었다(5-3).

7-6 전체 실행 목록 (batch 8 제외)

분석에 쓴 실행 28개다. 검증은 공식 검증 mAP75_95이고, Kaggle은 Public 점수다.

묶음	실행 이름	모델	데이터 · 방법	검증	Kaggle	분
A	1026_modelB_frcnn_v1_compare	Faster R-CNN	v1, 20 epoch	0.73839	–	3.4
A	1026_dataB_yolov8s_v1_compare20	YOLOv8s	v1, 20 epoch	0.80109	–	0.7
B	1102_yolov8s_baseline	YOLOv8s	v0 기본	0.93319	–	34.4
B	1026_dataB_yolov8s_v0_rfs01	YOLOv8s	v0 RFS 0.1	0.96106	–	52.2
B	1028_dataB_yolov8s_v1_clean	YOLOv8s	v1 라벨 정제	0.97547	–	22.5
B	1102_yolo11s_baseline	YOLO11s	v0 기본	0.94739	–	24.7
B	1102_yolo11s_v2_copypaste	YOLO11s	v2 Copy-Paste	0.95807	0.40090	43.1
B	1102_yolo26s_baseline	YOLO26s	v0 기본	0.94894	–	23.4
B	1031_sun_yolo26s_v0_d0	YOLO26s	v0 기본 재실행	0.92945	–	28.8
B	1026_dataB_yolov8s_v0_rfs01 *	YOLO26s	v0 RFS 0.1	0.96928	0.39833	44.8
B	1027_dataB_yolov8s_v2_copypaste *	YOLO26s	v2 Copy-Paste	0.94284	0.40500	48.5
B	1028_dataB_yolov8s_v1_clean *	YOLO26s	v1 라벨 정제	0.94801 (43장)	0.40138	28.3
B	1101_dataB_yolov26s_aihub	YOLO26s	v3 AI Hub	0.95016	0.41806	–
B	1101_yolov8s_v3all_aihub *	YOLO26s	v3all 새 클래스	0.96188	0.54422	226.1
C	1103_yolo26s_v3allc_d123_yolo26s_d1_e50	YOLO26s	v3allc RFS (d1)	0.98833	–	35.8
C	1103_yolo26s_v3allc_d123_yolo26s_d2_e50	YOLO26s	v3allcp CP (d2)	0.98315	–	19.9
C	1103_yolo26s_v3allc_d123_yolo26s_d3_e50	YOLO26s	v3allc 안전 증강 (d3)	0.97775	–	19.9
C	**1103_yolo26s_v3allc_d123_yolo26s_d12_e50**	**YOLO26s**	v3allcp RFS + CP (d12)	**0.99094**	미제출	35.8
C	1103_yolo26m_v3allc_d123_yolo26m_base_e50	YOLO26m	v3allc 기본	0.98737	–	35.0
C	1103_yolo26m_v3allc_d123_yolo26m_d1_e50	YOLO26m	v3allc RFS (d1)	0.99137	–	63.5
C	1103_yolo26m_v3allc_d123_yolo26m_d2_e50	YOLO26m	v3allcp CP (d2)	0.98461	–	35.0
C	1103_yolo26m_v3allc_d123_yolo26m_d3_e50	YOLO26m	v3allc 안전 증강 (d3)	0.96966	–	34.8

묶음	실행 이름	모델	데이터 · 방법	검증	Kaggle	분
C	1103_yolo26m_v3allc_d123_yolo26m_d12_e50	YOLO26m	v3allcp RFS + CP (d12)	0.99593	0.59671	61.5
D	1105_pegstar_yolo26s_v3allcp_kfold_v3allcp_f0_s42	YOLO26s	fold 0 (d12)	0.95901	–	39.4
D	1105_pegstar_yolo26s_v3allcp_kfold_v3allcp_f1_s42	YOLO26s	fold 1 (d12)	0.94061	–	30.5
D	1105_pegstar_yolo26s_v3allcp_kfold_v3allcp_f2_s42	YOLO26s	fold 2 (d12)	0.95134	–	33.8
D	1105_pegstar_yolo26s_v3allcp_kfold_v3allcp_f3_s42	YOLO26s	fold 3 (d12)	0.93549	–	31.7
D	1105_pegstar_yolo26s_v3allcp_kfold_v3allcp_f4_s42	YOLO26s	fold 4 (d12)	0.92736	–	29.4

* 실행 이름은 yolov8s로 시작하지만 설정 파일의 모델은 YOLO26s다. v3allc 기본을 30 epoch로 학습한 두 실행(YOLO26s Kaggle 0.59408, YOLO26m 0.60981)은 7-4 Kaggle 표에 따로 적었다.

각주와 출처

본문에서 뺀 각주와 근거 표기를 한곳에 모았다. "위치"는 최종 보고서의 절, 표, 그림 번호다.

각주

번호	위치	내용
1	1-1 표	65세 이상 만성질환 통계는 『한국의 사회동향 2024』(2023년 기준)와 「2023년 노인실태조사」에서 가져왔다.
2	4장	EDA 수치는 가이드 기준으로 다시 정리한 EDA 노트북(01_eda_.ipynb)에서 가져왔다.
3	4-6 표	D1~D11 구성은 01_eda_.ipynb, D6 외부 데이터 수치는 01_eda_external.ipynb 기준이다. D0(원본 기준)은 EDA 대상이 아니라 비교 기준이라 표에서 뺐다.
4	4-6 상자	교정 전후 재채점은 01_eda_.ipynb의 D10과 팀 W&B 기록(2026-11-02) 기준이다. 교정표(manual_fix)만 반영했고 제외 목록(manual_drop)은 넣지 않았다.
5	5-2 표	v3allc는 v3all에 라벨 정제 강화(D5)와 틀린 라벨 수정(D10)을 더한 버전이다(실험 이름의 d5_d10_d6).
6	그림 5-1	Kaggle 점수는 팀 제출 기록(2026-11-05 Public) 기준이다. batch 8로 학습한 실험(YOLOv8m 0.41373, 1024px YOLO26s 0.36800)은 뺐다.
7	5-3 상자	59개 클래스 매핑 문제는 11/01 제출 작업 기록과 고민분석_다음실험_웹앱계획.md 기준이다.
8	그림 5-2, 5-3	최종 학습 결과는 yolo26s_final.zip(단일 모델과 5-fold의 metrics.json, config.yaml, kfold_summary.csv, ENSEMBLE_SETTINGS.md) 기준이다. 5-fold 표준편차는 다섯 fold의 표본 표준편차다.
9	6-1 표	재실행 흔들림 0.0195는 Mac(mps)에서 같은 YOLO26s 설정을 두 번 돌린 결과(0.94894, 0.92945)의 차이다.
10	6-4 표	표의 값은 표준편차다. seed만 바꾼 경우와 fold를 바꾼 경우를 비교했다.
11	9-1 표	검증 47장, 알약 156개를 신뢰도 0.5 이상 예측으로 셌다. "최저 신뢰도"는 맞힌 약 중 가장 낮은 신뢰도다. 예측 파일은 preds_val.json이다.
12	9-3 표	테스트는 YOLO11s Copy-Paste와 YOLO26s AI Hub(v3), 검증은 YOLO11s Copy-Paste와 YOLO26s 기준을 비교했다. 예측 파일은 preds_val.json, preds_test.json이다.
13	전체	검증 점수는 실험별 metrics.json 기준이고, 모든 점수는 팀 공통 채점 함수 compute_map(신뢰도 0.001, NMS 0.5, 이미지당 최대 4개, IoU 0.75~0.95)으로 냈다.

출처

1. 통계청 통계개발원, 『한국의 사회동향 2024』, 2024.

2. 보건복지부·한국보건사회연구원, 「2023년 노인실태조사」, 2024.

3. 코드잇 AI 15기 Kaggle 대회 데이터 (학습 232장, 테스트 842장). 재배포가 금지되어 보고서에는 숫자와 그래프만 실었다.

4. AI Hub, 「경구약제 이미지 데이터」 (서울특별시 보라매병원 구축, 전체 2,793,978장). 대회 원본과 겹치는 데이터는 쓰지 않았다.

5. Ultralytics 라이선스 (AGPL-3.0 / Enterprise).

8. 최종 모델과 성과

8-1 두 가지 "최고" 모델

검증 기준 최고와 Kaggle 기준 최고가 서로 다른 실험이다.

항목	Kaggle 기준 최고	검증 기준 최고
실험 이름	1103_yolo26m0_d5_d10_d6_26m_yolo26m_v3allc_base_e30	1103_yolo26s_v3allc_d123_yolo26s_d12_e50
모델	YOLO26m	YOLO26s
데이터	v3allc 기본	v3allc + Copy-Paste 합성(v3allcp) + RFS 0.1
학습	30 epoch	50 epoch, 35.76분 (Windows GPU 1장)
Kaggle Public	**0.60981**	미제출
공식 검증 mAP75_95	– (같은 설정 50 epoch는 0.98737)	**0.99094**
5-fold 재검증	–	**0.943 ± 0.013** (5-3)
검증 알약 식별	– (50 epoch는 156/156, 최저 0.69)	156/156, 사진 47/47장, 최저 신뢰도 0.82

서비스 모델은 작은 YOLO26s로 계획한다. 같은 데이터(v3allc 기본, 30 epoch)에서 26s는 Kaggle 0.59408로 26m(0.60981)과 0.016 차이다. 검증에서 잰 재실행 흔들림(0.019)보다 작은 차이로, 더 가볍고 빠른 모델을 쓸 근거가 된다. 26s의 최종 설정은 5-fold에서도 0.943 ± 0.013으로 안정적이었다.

8-2 성과 요약

지표	값	의미
Kaggle Public 팀 최고	**0.60981**	처음 보는 사진 842장 기준 (YOLO26m)
같은 YOLO26s의 데이터 개선 폭	**+0.196** (0.398 → 0.594)	모델이 아니라 데이터로 얻은 개선
26s와 26m 차이 (v3allc 기본)	0.016 (0.594 vs 0.610)	작은 모델로도 거의 같은 점수
공식 검증 최고 (YOLO26s)	0.99094	아는 약에 대한 정확도. 누수가 있어 개선 폭은 Kaggle로 판단

지표	값	의미
5-fold 재검증 (YOLO26s 최종 설정)	0.943 ± 0.013	검증 사진을 바꿔도 흔들림(0.019)보다 작게 움직인다
검증 알약 식별	156/156, 최저 신뢰도 0.82	"확인 필요" 기준(0.5)보다 0.32 여유
재학습 시간	35.76분 (26s, 50 epoch) · fold별 29~39분	새 약을 추가해도 같은 날 다시 학습하고 배포 가능

해석할 때 주의할 점

공식 검증셋은 학습 사진과 같은 조합이 많아 점수가 부풀려져 있다(6-2). 0.99094와 156/156은 "아는 약에 대한 안정성"으로 읽고, 검증 사진을 바꿔 잰 5-fold 평균 0.943을 더 현실적인 검증 점수로 본다.

처음 보는 약에 대한 성능은 Kaggle 0.6 수준이다. 출시 전에는 직접 찍은 실사용 사진으로 다시 잰다(10장).

9. 오분류 분석

> **분석 방법**
>
> 검증 이미지 파일명에는 그 사진에 든 약 코드가 모두 들어 있다. 이것을 "사진 속 실제 약"으로 보고, 모델이 신뢰도 0.5 이상으로 예측한 약 이름과 비교했다. 대회 이미지는 쓰지 않고 숫자 표와 그래프로만 보여 준다.

9-1 검증셋에서는 이름은 맞히고, 박스 정밀도에서 깎인다

모델 (데이터)	이름 맞힘	사진	최저 신뢰도	놓친 약
YOLOv8s 기준 (v0)	156/156	47/47	0.64	–
YOLO11s Copy-Paste (v2)	155/156	46/47	0.77	일양하이트린정
YOLO26s 기준 (v0)	155/156	46/47	0.60	카나브정
YOLO26s 새 클래스 (v3all)	156/156	47/47	0.70	–
YOLO26s RFS (v3allc d1)	156/156	47/47	0.80	–
YOLO26s Copy-Paste (d2)	155/156	46/47	0.88	아토젯정
YOLO26s 안전 증강 (d3)	155/156	46/47	0.52	리피로우정
YOLO26s RFS + Copy-Paste (d12)	**156/156**	**47/47**	**0.82**	–
YOLO26m 기본 (v3allc)	156/156	47/47	0.69	–
YOLO26m RFS (d1)	156/156	47/47	0.90	–
YOLO26m Copy-Paste (d2)	156/156	47/47	0.87	–
YOLO26m 안전 증강 (d3)	156/156	47/47	0.75	–
YOLO26m RFS + Copy-Paste (d12)	156/156	47/47	0.83	–

검증 47장, 알약 156개를 신뢰도 0.5 이상 예측으로 셌다. "최저 신뢰도"는 맞힌 약 중 가장 낮은 신뢰도다.

모든 모델이 검증 알약을 거의 다 맞히는데 검증 mAP75_95는 0.93~0.99로 다르다. 점수 차이는 이름이 아니라 박스 경계를 IoU 0.85~0.95 수준으로 맞추는 정밀도에서 난다. v3allc 실험 9개에서 마지막 검증 box loss와

점수의 상관계수는 −0.93이었다. v3allc 실험 중 큰 모델(26m)은 다섯 실험 모두 156개를 맞혔고, 26s는 RFS를 쓴 d1과 d12만 모두 맞혔다.

9-2 클래스별 점수: 닮은 약이 있는 약에서 깎인다

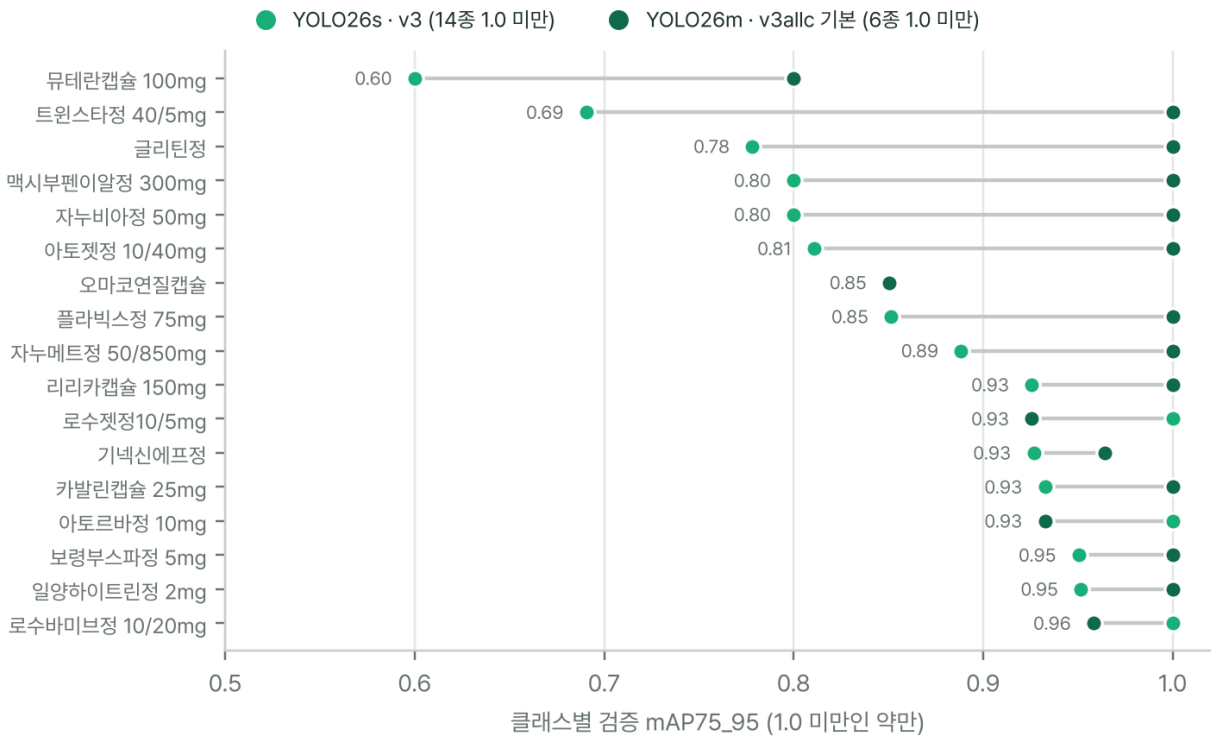


그림 9-1. 검증 점수가 1.0 미만인 약. YOLO26s(v3)에서는 14종, YOLO26m(v3allc 기본)에서는 6종이 1.0 미만이다. 왼쪽 숫자는 두 모델 중 낮은 점수다.

- 점수가 1.0 미만인 약은 대부분 색과 모양이 같은 "닮은꼴" 그룹에 속한다(26s v3 기준 14종 중 확인된 11종).
- 뮤테란캡슐은 같은 초록·장방형인 기넥신에프정(박스 45개)과, 자누비아정은 같은 주황·원형인 일양하이트린정(박스 153개)과 짝이다. **많은 약과 닮은 드문 약이 가장 불리하다.**
- 뮤테란캡슐은 26m의 모든 실험에서도 0.8로 가장 낮다. 모델을 키워도 해결되지 않는 약이다.

9-3 처음 보는 사진에서는 확신하며 틀린다

지표	검증 (아는 조합)	테스트 (처음 보는 사진)
신뢰도 0.5 이상 예측의 비율	82.9~83.0%	89.5~95.1%
두 모델의 답이 사진 단위로 갈리는 비율	4% (47장 중 2장)	24% (4장 중 1장꼴)

비교한 두 모델은 테스트에서 YOLO11s Copy-Paste와 YOLO26s AI Hub(v3), 검증에서 YOLO11s Copy-Paste와 YOLO26s 기준이다.

테스트에서도 모델은 검증만큼, 또는 더 높은 신뢰도로 답한다. 그런데 Kaggle 점수는 그보다 훨씬 낮다. 처음 보는 약을 아는 약으로 확신하며 잘못 맞히는 것으로 보인다. 그래서 신뢰도 숫자 하나로는 "모르는 약"을 걸러낼 수 없고, 여러 모델의 답이 갈릴 때 "확인 필요"를 띄우는 방식이 필요하다.

9-4 원인과 대응

원인	근거	대응
닮은 생김새	1.0 미만 약 대부분이 닮은꼴 그룹	"비슷한 약 있음" 경고, 매핑을 고친 뒤 각인 OCR 재검증
많은 약과 닮은 드문 약	뮤테란캡슐–기넥신에프정, 자누비아정–일양하이트린정	드문 약 데이터 보강 (외부 데이터, 직접 촬영)
학습에 없는 약	테스트에서 신뢰도는 높지만 점수는 낮음	모델끼리 답이 갈리면 "확인 필요"
박스 정밀도	box loss와 점수 상관 −0.93	박스 라벨 품질 개선, box 손실 가중치 실험
라벨 누락	정답이 빠진 사진 8장 (그중 4장 일양하이트린정)	빼지 않고 빠진 라벨을 추가

10. 배포 계획

YacssoK(약쏙)의 목표는 점수가 좋은 모델이 아니라 실제 사용자가 쓰는 서비스다. 이 장은 지금 동작하는 데모에서 출시까지의 경로와, 단계마다 다음으로 넘어가는 기준을 정리한다.

10-1 서비스 구조

순서	단계	내용
1	입력 검증 (Gradio)	사진 형식과 크기 확인, 촬영 안내
2	전처리	크기 맞춤(letterbox), RGB, 0~1 정규화
3	**YOLO26s 추론**	알약 위치, 약 번호, 신뢰도
4	후처리	겹친 박스 정리(NMS 0.5), 이미지당 최대 4개
5	약 정보 연결	약 번호 → 약 이름, 회사, 외형 정보 표
6	화면 표시	박스, 이름, 신뢰도, "확인 필요"와 안내 문구

YOLO26은 원래 NMS 없이 최종 박스를 내도록 설계된 모델이지만, 지금 파이프라인은 대회 채점 조건에 맞춰 NMS 0.5를 한 번 더 적용한다. 배포 버전에서 이 단계를 빼도 점수가 같은지 확인하고, 같다면 빼서 이식할 코드를 줄인다.

10-2 왜 YOLO26s인가

서비스 모델은 "가장 높은 점수"보다 "점수에서 뒤지지 않으면서 가장 싸고 빠르게 학습하고 배포할 수 있는 모델"로 골랐다. 모델 선택 근거는 7-5, 실행별 비교는 7장에 있다.

근거	배포에서의 의미
26m과 Kaggle 차이 0.016	같은 데이터에서 0.594 vs 0.610. 큰 모델을 쓸 이득이 작다
작은 모델 (약 950만 파라미터, 20.9 GFLOPs)	CPU 서버와 기기 안 실행의 비용이 낮다
빠른 재학습 (50 epoch 35.8분, 5-fold에서도 29~39분)	새 약을 추가하거나 라벨을 고친 뒤 같은 날 다시 학습하고 배포할 수 있다. 26m은 61.5분

근거	배포에서의 의미
1단계 구조	전처리, 모델, 약 정보 표 세 부분만 옮기면 서버·브라우저·휴대폰 어디서나 같은 결과를 낸다
신뢰도 여유 (검증 최저 0.82)	"확인 필요" 기준(0.5)보다 높아 아는 약에서 불필요한 경고가 적다
재현 가능한 학습 환경	uv로 고정한 환경, 설정 파일 하나로 정의한 실험, 같은 채점 함수. 재학습 모델을 이전 모델과 바로 비교할 수 있다

10-3 단계별 배포 로드맵

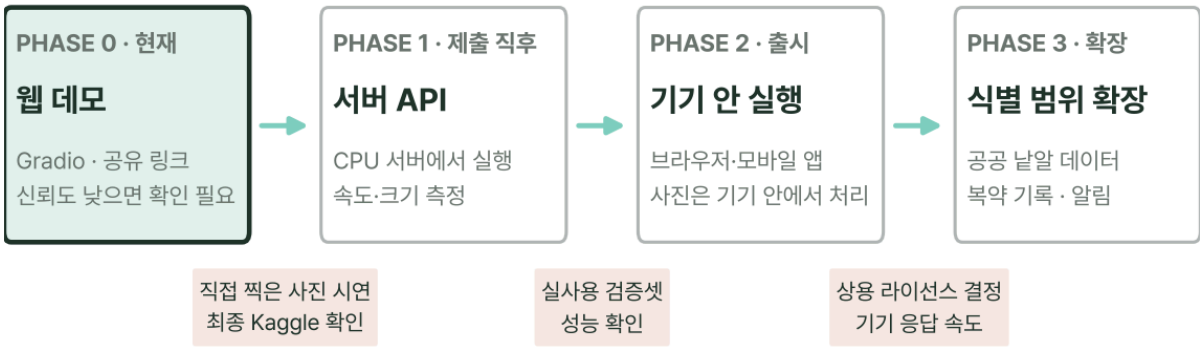


그림 10-1. 단계별 배포 로드맵. 분홍 칸은 다음 단계로 넘어가는 기준이다.

단계	할 일	넘어가는 기준
PHASE 0 · 현재 웹 데모	Gradio 웹 데모, 비밀번호로 보호한 공유 링크로 시연. 신뢰도가 낮으면 "확인 필요" 표시	직접 찍은 사진으로 시연 성공, 최종 모델 Kaggle 확인
PHASE 1 · 제출 직후 서버 API	학습한 모델을 CPU 서버에서 실행하는 API로 제공. 추론 속도와 모델 크기 측정. 휴대폰으로 찍은 실사용 검증셋 구축	실사용 검증셋 성능 확인
PHASE 2 · 출시 기기 안 실행	브라우저와 모바일 앱에서 모델 실행. 사진이 기기 밖으로 나가지 않음. "확인 필요" 사진만 서버의 큰 모델로 다시 확인	상용 라이선스 결정, 기기 응답 속도 확인
PHASE 3 · 확장 식별 범위 확장	공공 낱알식별 데이터로 약 종류 확대, 각인 OCR 재도입, 복약 기록과 알림, 보호자 공유	약을 늘려도 기존 약 성능 유지

10-4 서비스 지표

지표	현재	배포 기준
처음 보는 사진 (Kaggle Public)	0.60981 (26m) · 0.59408 (26s)	모델을 바꿀 때마다 함께 비교한다
공식 검증 mAP75_95	0.99094 (26s d12)	재학습 모델이 0.019 넘게 떨어지면 배포하지 않는다
5-fold 검증 평균	0.943 ± 0.013 (26s 최종 설정)	재학습할 때 같은 방식으로 다시 잰다
검증 알약 식별	156/156, 최저 신뢰도 0.82	실사용 검증셋에서 다시 잰다
추론 시간 (CPU, 장당)	측정 전	측정 후 목표를 정한다
모델 파일 크기	측정 전	기기 배포 가능 여부 판단
"확인 필요" 표시 비율	측정 전	실사용 사진으로 기준값을 정한다

10-5 책임 있는 서비스를 위한 장치

- 신뢰도가 낮은 결과는 "확인 필요"로 표시한다 (기준값은 실사용 검증셋으로 정한다).
- 여러 모델의 답이 갈리는 사진은 "확인 필요"로 표시한다 (테스트에서 24%).
- 닮은꼴 그룹에 속한 약은 "비슷한 약이 있음"을 함께 알린다.
- "참고용이며 복약 판단은 약사·의사와 상의하세요" 안내 문구를 띄운다. 진단·처방 기능은 넣지 않고 식별 보조로 한정한다.
- 출시 전 의료기기 소프트웨어 해당 여부를 확인한다.
- 사진이 기기 밖으로 나가지 않는 기기 안 실행을 출시 목표로 한다 (Phase 2).
- 홍보와 데모에는 대회 이미지가 아닌 직접 찍은 사진만 쓴다.

11. 한계와 리스크

11-1 기술적 한계

1. 공식 검증셋에 학습과 같은 조합이 섞여 있어 검증 점수가 실제보다 높게 나온다.
2. 학습에 없는 약은 알아볼 수 없고, 높은 신뢰도로 잘못 맞힐 수 있다 (Kaggle 약 0.6).
3. Mac(mps)에서는 같은 실험도 0.019 흔들린다. Windows GPU의 흔들림은 아직 재지 않았다.
4. 대부분의 실험을 한 번씩만 돌려 통계적으로 확정하기 어렵다.
5. 학습 사진은 같은 조합을 각도만 바꿔 찍은 것이 많아, 실제 사용자 사진(다양한 배경, 조명, 손 떨림)에서는 성능이 떨어질 수 있다.
6. 각인 OCR 파이프라인을 완성하지 못했다.

11-2 사업 리스크와 대응

리스크	대응
Ultralytics AGPL-3.0 라이선스	상용 출시 전 라이선스 방식(소스 공개 또는 유료 라이선스 등)을 정한다
대회 데이터 재배포 금지	데모와 홍보에는 직접 찍은 사진만 쓴다
잘못 알아본 결과로 인한 복약 사고	신뢰도·모델 불일치 표시, 비슷한 약 경고, 안내 문구, 약사 확인 유도
알아볼 수 있는 약의 범위	공공 낱알식별 데이터로 확장(Phase 3). 36분 재학습으로 빠르게 반영
의료 관련 규제	식별 보조로 기능을 한정하고, 출시 전 해당 여부를 확인한다

출처

통계 자료

1. 통계청 통계개발원, 『한국의 사회동향 2024』, 2024. (65세 이상 만성질환 2개 이상 63.9%, 평균 2.2개)
2. 보건복지부·한국보건사회연구원, 「2023년 노인실태조사」, 2024. (연령별 평균 만성질환 수)

데이터

- 코드잇 AI 15기 Kaggle 대회 데이터 (학습 232장, 테스트 842장). 재배포가 금지되어 이 보고서에는 숫자와 그래프만 싣는다.
- AI Hub, 「경구약제 이미지 데이터」 (서울특별시 보라매병원 구축, 전체 2,793,978장). 외부 학습 데이터로 썼고, 대회 원본과 겹치는 데이터는 쓰지 않았다.

분석 근거 파일

보고서 내용	근거
4장 EDA (4-1~4-6)	01_eda_.ipynb (D1~D11 구성)
4-6 D6 외부 데이터	01_eda_external.ipynb
4-6 교정 전후 재채점	01_eda_.ipynb D10, 팀 W&B 기록 (2026-11-02)
5장 Kaggle 점수	팀 제출 기록 (2026-11-05 Public 기준)
5-3 59개 클래스 매핑 문제	11/01 제출 작업 기록, 고민분석_다음실험_웹앱계획.md
7장 모델 비교	07_model_test.ipynb, rcnn_v1_compare20.yaml, yolo_v1_compare20.yaml (Faster R-CNN 비교), 실험별 metrics.json과 config.yaml (7-6의 28개 실행)
5-3 최종 학습 결과	yolo26s_final.zip (단일 모델과 5-fold의 metrics.json, config.yaml, kfold_summary.csv, ENSEMBLE_SETTINGS.md)
검증 점수 (5·6·8장)	실험별 metrics.json, 팀 공통 채점 함수 compute_map
9장 오분류	preds_val.json, preds_test.json (검증·테스트 예측 파일)

라이선스

- Ultralytics 라이선스 (AGPL-3.0 / Enterprise)